Web Scraping for Applied Research

Collecting and using data from web pages

Three approaches, compared, with a worked empirical example

Example: salary ranges in online job postings

A motivating example

Many US job postings now list a salary range, and some list a different range for each city the role is based in.

Same employer, same posting, different cities.
How much does the advertised range vary by location?

What we find (this sample)

- mean gap across cities: about **21%**
- roughly **a quarter** of pay variation between otherwise comparable postings

Administrative and survey data rarely record advertised pay at this level of detail, so the postings themselves are a useful complementary source.

What this means: Scraping is a practical option when relevant information is

Where we are going

Part 1 — Look first
Open the page. Find the numbers. Only then write code.

Part 2 — How the web works
Requests, replies, HTML and JSON.

Part 3 — Three ways to the same numbers
A hidden data address; parsing HTML; driving a browser.

Part 4 — From 3 jobs to 9,000
Loops, threads, caching, failures.

Part 5 — What the data says
An old question in labour economics.

Part 6 — Doing this responsibly

Everything runs from `slides/web-scraping/`. Start with **`python check_setup.py`**

Rule number one

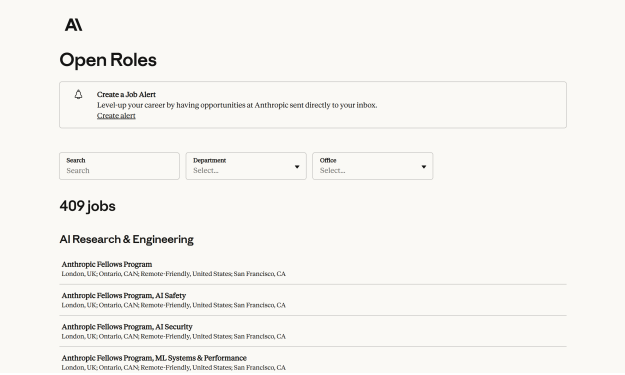
Never write a scraper for a page you have not read.

Before any code, answer three questions by looking:

1. **What exactly do I want?** Point at it on the screen.
2. **Where does it sit** in the page?
3. **How does the page get it?** Is it already in the page, or fetched separately?

What this means: Ten minutes of looking saves an afternoon of guessing.

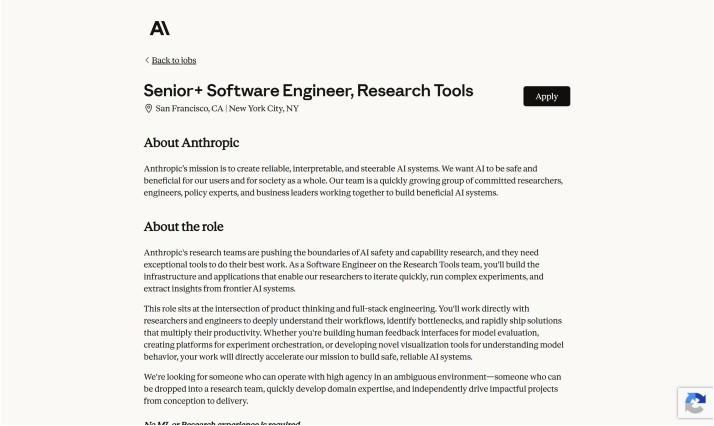
1. The board: every job at one company



`job-boards.greenhouse.io/anthropic`

Note. Postings shown here were live when the slides were prepared. Companies open and close positions continually, so the specific jobs and numbers may differ when you run the code.

2. One job advert



‹ Back to jobs

Senior+ Software Engineer, Research Tools

Apply

San Francisco, CA | New York City, NY

About Anthropic

Anthropic's mission is to create reliable, interpretable, and steerable AI systems. We want AI to be safe and beneficial for our users and for society as a whole. Our team is a quickly growing group of committed researchers, engineers, policy experts, and business leaders working together to build beneficial AI systems.

About the role

Anthropic's research teams are pushing the boundaries of AI safety and capability research, and they need exceptional tools to do their best work. As a Software Engineer on the Research Tools team, you'll build the infrastructure and applications that enable our researchers to iterate quickly, run complex experiments, and extract insights from frontier AI systems.

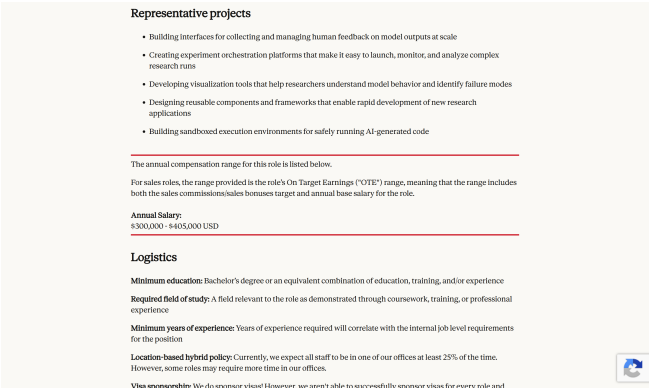
This role sits at the intersection of product thinking and full-stack engineering. You'll work directly with researchers and engineers to deeply understand their workflows, identify bottlenecks, and rapidly ship solutions that multiply their productivity. Whether you're building human feedback interfaces for model evaluation, creating platforms for experiment orchestration, or developing novel visualization tools for understanding model behavior, your work will directly accelerate our mission to build safe, reliable AI systems.

We're looking for someone who can operate with high agency in an ambiguous environment—someone who can be dropped into a research team, quickly develop domain expertise, and independently drive impactful projects from conception to delivery.

No ML or Research experience is required

`.../anthropic/jobs/4981828008`

3. Scroll down: there they are



Representative projects

- Building interfaces for collecting and managing human feedback on model outputs at scale
- Creating experiment orchestration platforms that make it easy to launch, monitor, and analyze complex research runs
- Developing visualization tools that help researchers understand model behavior and identify failure modes
- Designing reusable components and frameworks that enable rapid development of new research applications
- Building sandboxed execution environments for safely running AI-generated code

The annual compensation range for this role is listed below.

For sales roles, the range provided is the role's On Target Earnings ("OTE") range, meaning that the range includes both the sales commissions/sales bonuses target and annual base salary for the role.

Annual Salary:
$300,000 - $405,000 USD

Logistics

Minimum education: Bachelor's degree or an equivalent combination of education, training, and/or experience

Required field of study: A field relevant to the role as demonstrated through coursework, training, or professional experience

Minimum years of experience: Years of experience required will correlate with the internal job level requirements for the position

Location-based hybrid policy: Currently, we expect all staff to be in one of our offices at least 25% of the time. However, some roles may require more time in our offices.

Visa sponsorship: We do sponsor visas! However, we aren't able to successfully sponsor visas for every role and

What this means: Three things we want per job: the title, the location, and the two numbers.

Right-click → Inspect: where do the numbers live?

The browser shows you the page's structure. The salary sits inside a <bdi> tag:

```
<div class="pay-input">
  <bdi>$300,000</bdi>
  <span>-</span>
  <bdi>$405,000</bdi>
  <bdi>USD</bdi>
</div>
```

That tag name is all we need to find them later.

Try it

1. Open a job advert
2. Right-click on the salary
3. Choose **Inspect**
4. Read the tag it highlights

<bdi> is a real HTML tag ("bidirectional isolate"). It has nothing to do with money—the site just happens to use it here.

A useful habit: watch the page's own requests

A web page is not one download. It loads, then asks the server for more things: images, styles, and often **the actual data**.

See it happen

1. Press **F12**
2. Tab **Network**
3. Filter **Fetch/XHR**
4. Reload the page

One of the requests that appears:

```
boards-api.greenhouse.io/v1/boards/
  anthropic/jobs/4981828008
  ?pay_transparency=true
```

Open that address yourself and you get the same job—as **data**, not as a page.

What this means: The browser already retrieves this data to build the page. We can request it directly instead of parsing the rendered HTML.

The same job, as data


```
{
  "id": 4981828008,
  "title": "Senior+ Software Engineer, Research Tools",
  "location": { "name": "San Francisco, CA | New York City, NY" },
  "company_name": "Anthropic",
  "first_published": "2025-11-07T19:13:30-05:00",
  "pay_input_ranges": [
    {
      "min_cents": 30000000,
      "max_cents": 40500000,
      "currency_type": "USD",
      "title": "Annual Salary:"
    }
  ]
}
```


What this means: No parsing, no guessing. The salary is already a number—and it even tells us the currency.

Every scrape is the same two steps

1. Request

You ask a server for an address (URL). You may attach headers saying who you are.

2. Reply

The server sends back a status code and some content.

Status codes worth knowing

- `200` fine, here it is
- `403` I know who you are, and no
- `404` no such thing
- `429` you are asking too fast
- `500` the server broke

Always check the code. A 404 page still "downloads" perfectly happily, and will silently produce nonsense.

What this means: Scraping is just doing by hand what your browser does automatically.

Two kinds of reply

HTML — built for eyes

```
<div class="pay-input">
  <bdi>$300,000</bdi>
  <span>-</span>
  <bdi>$405,000</bdi>
</div>
```

Layout and content mixed together. You must dig the values out, and the digging breaks when the design changes.

JSON — built for programs

```
{
  "min_cents": 30000000,
  "max_cents": 40500000,
  "currency_type": "USD"
}
```

Names and values, nothing else. It becomes a Python dictionary in one line.

What this means: If a JSON version exists, use it. It is faster, cleaner, and far more stable.

The decision tree

1. **Is there a bulk download?** Take it and stop. (Statistical agencies, World Bank, Eurostat.)
2. **Is there an official API?** Use it. Documented and stable.
3. **Is there a hidden JSON address?** we are here Check the Network tab. Very often yes.
4. **Is the content in the HTML?** Then `requests` + `BeautifulSoup`.
5. **Does the page need a real browser?** Then Selenium. Slow, but it always works.
6. **Is the site actively blocking you?** Reconsider the question, not the tooling.

What this means: Work down the list. Most people start at step 4 and never discover that step 3 would have done.

Method 1: request the data directly

```python
import requests

url = ("https://boards-api.greenhouse.io/v1"
       "/boards/anthropic/jobs/4981828008"
       "?pay_transparency=true")

r = requests.get(url, timeout=60)
r.raise_for_status()        # stop if not 200

job = r.json()              # -> a Python dict
pay = job["pay_input_ranges"][0]
```

Output

```
job["title"]
  Senior+ Software
  Engineer, ...

pay["min_cents"]/100
  300000.0
pay["max_cents"]/100
  405000.0
```

What this means: No parsing: `.json()` returns a dictionary and we read fields from it.

Inspect live: `boards-api.greenhouse.io/v1/boards/anthropic/jobs/4981828008?pay_transparency=true`

Method 2: download the page, parse the HTML

```
import requests
from bs4 import BeautifulSoup

html = requests.get(url_web, timeout=60).text
soup = BeautifulSoup(html, "html.parser")

title = soup.find("h1").get_text(strip=True)

# each <bdi> tag holds a value
texts = [t.get_text(strip=True)
         for t in soup.find_all("bdi")]

amounts = [float(t.replace("$","")
           .replace(",",""))
           for t in texts if t.startswith("$")]
```

Output

```
texts
  ['$300,000',
   '$405,000',
   'USD']

amounts
  [300000.0,
   405000.0]
```

What this means: Works, but relies on the salary sitting in a <bdi> tag. If the page design changes, this can break without warning.

Method 3: drive a real browser

```python
from selenium import webdriver
from selenium.webdriver.chrome.options import Options
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

options = Options()
options.add_argument("--headless=new")      # run with no visible window
driver = webdriver.Chrome(options=options)

driver.get("https://job-boards.greenhouse.io/anthropic/jobs/4981828008")

WebDriverWait(driver, 20).until(            # <- do not skip this
    EC.presence_of_element_located((By.TAG_NAME, "bdi")))

texts = [e.text for e in driver.find_elements(By.TAG_NAME, "bdi")]
driver.quit()                               # always close it
```

What this means: Selenium is the tool that always works, and the one to reach for last.

When you actually need Selenium

Use it when

- the page is empty until JavaScript fills it
- you must click, scroll, or log in
- content appears only after interaction

The cost

- a whole browser per run
- slow, and heavy on memory
- breaks in new ways (see next slide)
- needs a matching driver

Good news: Selenium 4 downloads the right driver for you. Older guides tell you to install `chromedriver` by hand—you do not have to, provided you are on `selenium>=4.25`.

The error you will meet in your first ten minutes

```
selenium.common.exceptions.StaleElementReferenceException:
Message: stale element reference: stale element not found
```

What happened

You found an element. The page then rebuilt itself and threw that element away. Your variable now points at something that no longer exists.

The fix

Wait for the page to settle, then look the element up *again*.

```python
for attempt in range(3):
    try:
        title = driver.find_element(
            By.TAG_NAME, "h1").text
        break
    except StaleElementReferenceException:
        time.sleep(1)
```

What this means: This is not your bug. It is what browsers do. Write for it from the start.

The three methods return the same values

Posting (software engineering)	Lower	Upper	Range width
Senior+, Research Tools	$300,000	$405,000	29.8%
Staff, Labs: Applied AI	$320,000	$405,000	23.4%
Staff+, Platform	$405,000	$485,000	18.0%

Identical across all three methods (JSON, BeautifulSoup, Selenium). Range width = (upper − lower) / midpoint.

What this means: The choice of method affects cost and robustness, not the result.

Cost per method (three postings)

Method	Total (3)	Per posting	Relative
JSON endpoint	1.4 s	0.45 s	1.0×
requests + BeautifulSoup	1.9 s	0.64 s	1.4×
Selenium (browser)	2.1 s	0.71 s	1.6×

Median of three runs; timings vary with network conditions.

Differences are small at this scale. They matter much more once we collect thousands of postings (next slide).

The larger gain: request many records at once

The board endpoint returns *every open job at the company* in one request. Page-by-page methods cannot: one page is one job.

Approach	Time for one company (about 400 postings)	Relative
Board endpoint (one request)	∼**1 s**	1×
JSON, one posting at a time	∼3 min	∼170×
BeautifulSoup, per page	∼4–5 min	∼235×
Selenium, per page	∼5 min	∼260×

What this means: Most of the running time is network round-trips, not parsing. Fewer requests is the main lever.

From 3 jobs to 9,000: four new problems

1. Things fail

Boards close, names change, connections drop. One failure must not kill the run: catch the error, return empty, carry on.

2. Waiting dominates

Your program is idle almost all the time. Run several requests at once with a thread pool.

3. Re-running wastes everything

Save every raw reply to disk. The second run costs nothing and makes no new requests.

4. You are a guest

Identify yourself. Keep the number of workers modest. Take the bulk endpoint when it exists.

What this means: Caching raw replies is also what makes your results reproducible six months later.

All four, in twenty lines

```python
def fetch_board(company, use_cache=True):
    path = f"data/cache/{company}.json"

    if use_cache and os.path.exists(path):      # 3. cache
        return json.load(open(path))

    try:                                        # 1. never crash the run
        r = SESSION.get(BOARD_API.format(company=company), timeout=45)
        if r.status_code != 200:
            return []
        jobs = r.json().get("jobs", [])
    except Exception:
        return []

    json.dump(jobs, open(path, "w"))
    return jobs

with ThreadPoolExecutor(max_workers=8) as pool:  # 2. several at once
    results = pool.map(fetch_board, companies)
```

What came back

Company boards requested	109
of which returned data	63
Postings collected	8,852
with a salary range	4,022 (45%)
listing > 1 pay zone	583

Collected in about 10 seconds. Figures depend on which postings are live at run time.

Many boards returned nothing—companies close boards and change names. Handling this is why error handling matters.

Under half disclose a salary. Disclosure is shaped by state pay- transparency laws, which could itself be studied.

Background: compensating differentials

Compensating differential theory (Smith; Rosen, 1986): in equilibrium, undesirable job attributes should be associated with higher pay, desirable ones with lower pay.

Estimating this from wage data is difficult. A common concern: more productive workers tend to obtain both higher pay *and* better working conditions, and worker productivity is not fully observed. Estimated differentials can therefore come out with an unexpected sign.

Why postings are a useful complement. A posted range is attached to a vacancy, not to a hired worker, so worker selection does not enter the observation in the same way.

Result 1: variation by location, within a posting

Some postings list several pay zones—one posting, several salary bands by city. Only location differs.

Postings with > 1 pay zone	574
Top-to-bottom gap, mean	20.7%
median	14.2%
90th percentile	28.5%
Var. of log pay within a posting (location)	0.013
Var. within company $\times$ family $\times$ level	0.050
share attributable to location	$\approx 25\%$

What this means: About a quarter of pay variation between otherwise comparable postings is associated with location alone.

Result 2: the remote-work pay differential

Regression of log posted pay on a remote indicator, under two sets of controls.

	(1)	(2)
Remote (% difference)	+9.3	+1.2
	(2.6)	(0.5)
Comparison within	firm × family × seniority	firm × exact title
Observations	4,450	129
Cells	413	22

Cluster-robust SEs (by company) in parentheses.

What this means: The estimate shrinks toward zero under a tighter comparison: much of column (1) reflects which roles are offered remotely, not a pay premium.

A caution before you believe any of this

What this sample is

- firms using one hiring platform
- mostly technology, mostly high-paying
- only those disclosing a salary
- *advertised* pay, not paid pay

What follows

- this is not "the labour market"
- disclosure is a choice, and choices select
- the second estimate rests on 22 comparisons

What this means: Scraped data is real data with real limits. Say what your sample is before you say what you found.

Scraping responsibly

Practical

- prefer a bulk download or an API
- send a `User-Agent` saying who you are
- ask for many records per request
- keep concurrency modest; back off on `429`
- cache raw replies

Legal and ethical

- read `robots.txt` and the terms
- public $\neq$ free of obligations
- do not collect personal data you do not need
- GDPR applies to people, not to job adverts
- never scrape behind a login you agreed not to

What this means: Keeping raw responses reduces load on the server and makes your results reproducible.

Your turn

Exercise

1. Add ten companies of your choice to `COMPANIES` in `05_scale_up.py` and re-run it.
2. Pick a job feature the adverts mention—seniority, department, number of locations, how long the advert has been open.
3. Ask whether posted pay differs along that feature, comparing *within* the same company.
4. Write one paragraph: what you found, and one reason it might be wrong.

What this means: Step 4 is the one that matters.

What to remember

1. **Look at the page before writing code.** Always.
2. **Check for a hidden JSON address** before parsing any HTML.
3. **Fewer requests**, not faster parsing.
4. **Expect failure** and write so one failure costs you nothing.
5. **Cache the raw replies.**
6. **Collecting data is often the easy part**—the comparison you choose is the substantive work.

All code, data and this deck:
`github.com/ChristosMylonakisCEMFI/CSS-DataScience`

Appendix

Appendix: the false start (this really happened)

First attempt: find the salary with a regular expression in the page text.

```
pat = re.compile(r"\$\s?(\d{1,3}(?:,\d{3})+)\s*-\s*\$\s?(\d{1,3}(?:,\d{3})+)")
```

Regex on the text
salary found for **13%** of postings
hourly vs yearly confusion, currencies, ranges written five ways

After adding
`?pay_transparency=true`
salary found for **45%** of postings
typed integers, explicit currency, one entry per pay zone

What this means: Look for structure before you write a regular expression. The missing 32 points were not in the text—they were in a field we had not asked for.

Appendix: politeness in code

```python
import time, requests

SESSION = requests.Session()                  # reuse the connection
SESSION.headers.update({"User-Agent": "CEMFI research (you@email.com)"})

def get(url, tries=4):
    for attempt in range(tries):
        r = SESSION.get(url, timeout=45)
        if r.status_code == 429:              # too fast: wait longer each time
            time.sleep(2 ** attempt)
            continue
        if r.status_code >= 500:              # server problem: try again
            time.sleep(1 + attempt)
            continue
        return r
    return None
```

What this means: Exponential backoff: wait 1s, then 2s, then 4s. Hammering a struggling server is how you get blocked.

Appendix: a session is faster than repeated `get`

```python
# new connection every time
for url in urls:
    requests.get(url)
```

Each call re-negotiates encryption with the server. On this site that roughly **doubled** the time per page.

```python
# one connection, reused
s = requests.Session()
for url in urls:
    s.get(url)
```

Also remembers headers and cookies, so you set them once.

We hit this while preparing the session: BeautifulSoup looked *slower than Selenium* until we used a `Session`. Selenium was reusing its connection; our `requests` code was not.

Appendix: common errors and what they mean

Error	Usually means
`ModuleNotFoundError`	installed for a different Python; run `check_setup.py -fix`
`ConnectionError` / timeout	network, proxy, or the server is slow—retry with backoff
`JSONDecodeError`	the reply was not JSON (often an error page); print `r.text`
`AttributeError: 'NoneType'`	`soup.find()` found nothing; the tag changed
`StaleElementReference`	the page rebuilt itself; wait and look again
`SessionNotCreated`	Selenium older than 4.25, or Chrome not installed
`HTTP 403`	the server refused you; check the terms before working around it
`HTTP 429`	you are going too fast; slow down

Appendix: where to go next

Tools

- `playwright` — a faster, more modern Selenium
- `scrapy` — a framework for large crawls
- `lxml` — a much faster HTML parser
- `polars` — pandas, but quick on big files

Data worth knowing about

- other hiring platforms expose boards the same way
- government portals: procurement, courts, transparency registers
- the Internet Archive turns any static site into a historical panel

The habit that generalises: open the page, press F12, watch the Network tab. You will be surprised how often the clean data is already there.